

VScode+Clineで効率化するリバースエンジニアリング ～仕様書作成編～

現在のプロジェクトについて

- PHP5.6系からPHP8.3系 + Laravel11へのリプレイス案件
- 開発の流れ：
 - リプレイス対象機能リストから機能を選択
 - 旧コードを読み込み仕様を洗い出し
 - チケット化してセブに開発依頼
- Amazon Bedrockを使う権限をもらったので、Cline(Claude3.7)を使いながら開発の効率化の真っ最中
 - 次はCopilot + Cursorに移行するみたい

Clineに期待したこと

1. 仕様の解析

- 現状一人で全メンバー分のチケット/仕様書作成をしており時間が圧迫されがち
- 仕様書作成の効率化ができれば開発参加や将来的なタスク作成が可能なる

2. 旧コードのリファクタリング

- 既存コードが読みづらい
 - 一つのHTMLファイル内でリクエストパラメータによって処理を分けていて全く違う処理が一つのHTMLファイル内に実装されている

```
if(isset($_POST['create'])) {  
    // 新規作成ページ用の処理  
} elseif(isset($_POST['createConfirm'])) {  
    // 新規作成確認ページ用の処理  
} elseif(isset($_POST['register'])) {  
    // 実際の登録処理  
} elseif...
```

- 開発者、セブレビュワー、自分の最低3人が同じコードを読むので、リファクタリングされたコードを共通して読めれば旧コードの理解度、読む時間が効率化される

Clineの初期設定

memory-bankとは

- Clineの最大の特徴：**セッション間で記憶が完全にリセットされる**
- Memory Bank：
 - **セッション間で知識を継続させる**ための構造化されたドキュメント群
 - 各セッション開始時に必ずMemory Bankを読み込む仕組みを設けることで、セッションが分かれていてもClineは別のセッションのコンテキストを持ったまま新しいセッションを始められる
 - プロジェクトの一貫性と継続性を保つための重要な基盤
 - 更新タイミングを指定することでCline自身の判断でMemory-Bankを更新してくれるようになる

Clineの初期設定

Memory Bankの設定方法

1. Custom Instructionsへの記載 (Clineの公式ドキュメントからコピペ)

Cline's Memory Bank

I am Cline, an expert software engineer with a unique characteristic: my memory resets completely between sessions. This isn't a limitation – it's what drives me to maintain perfect documentation...

- VSCodeのCline拡張機能に対して直接指示(local内で同じ設定が共有)

2. clinerules.mdファイルの作成

```
description: >
  このプロジェクトではadmin, fuelシステムの仕様調査、リプレイスに向けての設計を行っており、仕様調査/設計を効率的に行うためにAIツール（Cline）を活用する。
goal: >
  仕様調査/設計の効率化と、AIアシスタント活用による作業最適化。

memory_bank:
usage_policy:
  - セッション開始時にMemory Bankを必ず読み込む。
  - 初回は "projectbrief.md", "operation.md", "systemPatterns.md", "techContext.md" を読む。
directory_structure:
  projectbrief.md: プロジェクト概要
  ...
update_triggers:
  - 新仕様やパターン発見時
  - 重要な機能追加・仕様変更時
```

- リポジトリ内に作成するのでリポジトリ内で設定が共有

Clineの初期設定

memory-bankの活用

Memory Bankファイルの基本構成と役割

1. **projectbrief.md** - プロジェクトの目的、背景、主要機能、成功基準
2. **productContext.md** - ユーザーニーズ、課題、ユースケース、ビジネス目標
3. **systemPatterns.md** - アーキテクチャ、デザインパターン、コンポーネント関係
4. **techContext.md** - 使用技術、開発環境、依存関係、技術的制約
5. **activeContext.md** - 現在の作業、最近の変更、次のステップ、検討事項
6. **progress.md** - 完了タスク、進捗状況、残作業、タイムライン

Clineの初期設定

Memory Bankの更新タイミングと方法

- **新しいパターン発見時**：システム理解が深まった際に更新
- **重要な実装後**：コードベースに大きな変更を加えた後
- **「update memory bank」指示時**：ユーザーからの明示的な指示
- **コンテキスト明確化時**：プロジェクト方針や要件が変わった際

更新プロセス

1. 全ファイルの見直し
2. 現状の正確な文書化
3. 次のステップの明確化
4. 新たな知見やパターンの記録

Clineの初期設定

オリジナルファイルの作成

1. operation.md

- 用途：仕様調査の概要説明、プロセスの詳細手順書へのリンク
- 目的：Clineが求められているスコープを理解するため

指示パターン

仕様調査パターン

仕様調査は以下の手順で行います。この手順は、既存システムの機能を効率的に理解し、文書化するための標準的なアプローチです。

与えられた指示に基づいて適切な項目を参照してください。

ステップ2,3,4に関してはエンドポイントごとの影響範囲ごとに処理を行うので、機能全体のエンドポイントを理解した上で、指定されたエンドポイントに対しての処理を行ってください。

1. HTMLファイルの解析

[こちらのファイル](./operation/analyze0verview.md)を参照

2. 解析ディレクトリにファイルをコピー

[こちらのファイル](./operation/pickUpFiles.md)を参照

Clineの初期設定

オリジナルファイルの作成

2. operation/**.md

- 用途：仕様調査フローの詳細な手順書
- 目的：指示内容と期待するアウトプットの理解

URLからのエンドポイント分析

概要

渡されたHTMLファイルから関連するエンドポイントを特定、一覧化し、エンドポイント単位で仕様調査ができるようにドキュメントの作成とディレクトリの作成を行う

目的

adminでは1つのHTMLが複数のリクエストに対応していることがあり、条件分岐が多岐にわたることによって可読性が著しく低下している。そこで、HTMLファイルをエンドポイント単位に分けることによって可読性の向上、Clineが読み込むトーク数の削減を行いたい。その前段として、HTMLファイルを解析しエンドポイントを洗い出しを行い後続のエンドポイント単位の仕様調査をしやすいようにしたい

ステップ

1. URLからのエンドポイント分析

1. **対象URLの特定**:

- 調査対象の機能に関連するHTMLファイルを特定
- 例: `user\_list.html`, `user\_edit.html`

2. **コード検索**:

- `search\_files`ツールを使用して、対象ファイルを検索
- 関連するフォームのaction属性やリダイレクト先を特定するための正規表現を使用
- 例: `(action\s\*=\s\*["'](?:[^\s"]|\\")\*["']|header\s\*\(\s\*["'](?:[^\s"]|\\")\*["']Location:\s\*["'](?:[^\s"]|\\")\*["']\))`

仕様書作成の進め方

1. HTMLファイルからエンドポイント(機能)の洗い出し

ユーザー情報管理機能

このディレクトリには、管理システムにおけるユーザー情報管理機能に関するエンドポイントの仕様がまとめられています。

概要

ユーザー情報管理機能は、ユーザー情報を管理するための機能です。主に以下の2つのHTMLファイルで構成されています：

1. `user\_list.html` - ユーザー情報一覧画面
2. `user\_edit.html` - ユーザー情報編集/確認画面

エンドポイント一覧

この機能には、以下の10個のエンドポイントが含まれています：

1. [ユーザー情報一覧画面](./list/) - GET /user_list.html
2. [CSVダウンロード処理](./csv-download/) - POST /user_list.html (csvdl)
3. [CSVアップロード処理](./csv-upload/) - POST /user_list.html (change_upload)
4. [CSV一括処理](./csv-process/) - POST /user_list.html (do_decision_csv)
5. [ユーザー情報編集/確認画面](./edit-view/) - GET /user_edit.html
6. [ユーザー情報編集画面への遷移](./edit-update/) - POST /user_edit.html (update)
7. [編集モードへの切り替え](./edit-mode/) - POST /user_edit.html (edit)
8. [編集内容の確認](./edit-confirm/) - POST /user_edit.html (confirm)
10. [一覧画面への戻り](./return-to-list/) - POST /user_edit.html (return)

画面遷移

- user_list.html → user_edit.html: 一覧画面から編集画面への遷移
- user_edit.html → user_list.html: 編集画面から一覧画面への戻り

仕様書作成の進め方

2. 参照ファイル、参照している関数の洗い出し

参照ファイル

メインファイル

– `user\_edit.html` – パートナー情報の編集・表示を行うメインファイル

参照ファイル (admin)

2. `/var/www/libs/classes/User.class.php` – ユーザー関連クラス
3. `/var/www/libs/classes/UserMaster.class.php` – ユーザーマスタークラス
4. `/var/www/libs/classes/Items.class.php` – アイテムクラス（選択肢の生成など）
5. `/var/www/libs/classes/Tools.class.php` – ツールクラス（フォーム要素の生成など）
6. `/var/www/libs/classes/GenreCategory.class.php` – ジャンルカテゴリクラス

仕様書作成の進め方

3. 不要コードの削除 (リファクタリング)

- Clineに読み込ませるコンテキストを少なくするため、人間が読むときに不要なコードを削除するため

4. 実行されるSQLの解析

- Clineが仕様書を作成するときに参考にできるようにするため

```
/**
 * 全塾データを取得するためのメソッド
 *
 * @sql SELECT * FROM user;
 * @param obj $mysqli
 * @return obj
 */
public function select_all_user_data($mysqli) {
    ...
}
```


仕様書作成の進め方

5. 仕様書の作成

- 機能の概要、処理フロー、データ構造などを文書化
- 画面遷移や入出力の詳細を記述

CSV一括処理エンドポイント仕様書

1. 基本情報

```
- **URL**：`/admin/html/info_partner_list.html`
- **HTTPメソッド**：POST
- **パラメータ**：
  - POST
    - `do_decision_csv` - CSV処理実行のトリガー
    - `up_file_name` - アップロードされたCSVファイルのパス
- **機能概要**：CSVファイルを使用してパートナーの承認/非承認を一括で処理する機能。CSVの16列目に「Y」が入力されていれば承認、「N」が入力されていれば非承認として処理する。
- **処理フロー概要**：
  1. 初期データの取得
    - サイトジャンル情報の取得
    - サイト業種情報の取得
  2. CSV一括処理(`do_decision_csv`が設定されている場合)
    1. CSVファイルを開く
    2. 各行を処理
      1. 16列目の値が空または「Y」「N」以外の場合はスキップ
      2. 16列目の値が「Y」の場合、パートナーを承認
        1. パートナーマスターの更新
        2. パートナーサイトマスターの更新
        3. 変更履歴の記録
```

2. 処理のフロー詳細

2.1 初期データの取得

2.1.1 サイトジャンル情報の取得

```
- 関数名：`GenreCategory::getSiteGenre()`
- 処理内容：サイトジャンル情報を取得
- 戻り値：サイトジャンル情報の配列
- 処理詳細：
  - サイトジャンル情報を取得し、0番目の要素に空文字を設定
- 発行SQL：
  ```sql
 SELECT site_genre_id, site_genre_name
 FROM site_genre_master
 WHERE delete_flg = 0
 ORDER BY site_genre_id
```

# 現状のClineの評価

## 得意そうなこと

- 既存のコードから指定した特徴点を抜き出すこと
  - アウトプットのフォーマットが厳しくないため

## 不得意そうなこと

- 仕様書作成やリファクタリングなど期待されたアウトプットを出すこと
  - 細かく指示しないと期待通りのアウトプットが出ない
  - 期待値以上のことをやろうとする傾向
  - アウトプットのフォーマットを多様に考えすぎる

## 指示する上で意識したこと

- 期待するアウトプットのフォーマットがあるのであれば、例を指示と一緒に渡すと精度が上がる
- Memory Bankは常に全て読み込ませるのではなく、必須知識と特定ケースにおける知識を分離し、読み込ませるタイミングを管理することで、読み込むToken数を管理する

# 今後の展望

- 仕様書作成のさらなる時間削減

- 現状はClineと対話しながら細かくFBが必要で速度はそこまで出ていない
  - 今までのものよりもリッチな仕様書を出力しているため、一定の理解はできる

- リファクタリングの適切な範囲の見極め

- 時間効率向上のための事前処理の最適化
  - どれだけの量のコードを読み込ませると指示した内容の精度が落ちるのかの閾値を理解し、仕様書出力までの時間を早くする

- memory-bankの文章の改善

- 短い文章で高い精度を出せるよう最適化

- 設計パートへの進出

- 仕様書作成から設計までの一貫対応を目指す

## 参考資料

- 【Cline公式】 memory bankの導入方法